Module 6 — React: First App & Components

Time: 1 hour · Covers **Practical 5** · **CO3**

> *Practical 5: Create React First app & display Hello World with the help of components.*

Part A — What is React? (concept, ~15 min)

React is a JavaScript **library for building user interfaces** out of small, reusable pieces called **components**. Instead of manually updating the DOM, you describe *what the UI should look like* and React keeps the screen in sync.

Two core ideas:

1. **Components** — a function that returns UI. You compose big UIs from small components, like Lego bricks.
2. **JSX** — an HTML-like syntax you write *inside* JavaScript. It is not HTML; it compiles to function calls. `<h1>Hi</h1>` becomes `React.createElement(...)`.

```
// A component is just a function that returns JSX.
function Hello() {
  return <h1>Hello, World!</h1>;
}
```

Part B — Create the app with Create React App (~15 min)

Create React App (CRA) scaffolds a ready-to-run React project.

To create a brand new app from scratch (what the practical asks):

```
npx create-react-app my-first-app
cd my-first-app
npm start          # opens http://localhost:3000
```

This module already contains a CRA-style project so you can run it immediately:

```
cd 06-react-hello
npm install        # installs react + react-scripts (first time only)
npm start          # opens http://localhost:3000
```

Project structure

```
06-react-hello/
├── public/
│   └── index.html        # the single HTML page; React mounts into <div id="root">
├── src/
│   ├── index.js          # entry point: renders <App> into the page
│   ├── App.js            # the root component
│   ├── App.css           # styles
```

```
│   └── components/
│       ├── Greeting.js   # a reusable component
│       └── WelcomeCard.js# a component that COMPOSES others
└── package.json
```

Part C — How a React app boots (~15 min)

Follow the chain:

1. **`public/index.html`** has an empty `<div id="root"></div>`. That is the mount point.
2. **`src/index.js`** finds `#root` and renders the `<App />` component into it.
3. **`src/App.js`** returns JSX that includes smaller components.

```
// src/index.js — the entry point
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";

const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(<App />); // mount the whole app
```

Read `src/index.js` → `src/App.js` → `src/components/Greeting.js`.

Part D — Writing components (~15 min)

A simple component

```
// src/components/Greeting.js
function Greeting() {
  return <h1>Hello, World!</h1>;
}
export default Greeting;
```

Composing components

Components can use other components, building a tree:

```
// src/components/WelcomeCard.js
import Greeting from "./Greeting";

function WelcomeCard() {
  return (
    <div className="card">
      <Greeting />                {/* reuse the Greeting component */}
      <p>Welcome to your first React app.</p>
    </div>
  );
```

```
}
export default WelcomeCard;
```

Note `className` (not `class`) — because `class` is a reserved word in JavaScript, JSX uses `className` for CSS classes.

You can also embed JavaScript expressions in JSX with `{ }`:

```
const name = "Ada";
return <h1>Hello, {name}!</h1>; // renders "Hello, Ada!"
```

Summary

- React builds UIs from **components** (functions that return **JSX**).
- **Create React App** scaffolds a runnable project; `npm start` runs it.
- Boot chain: `index.html` (#root) → `index.js` (render) → `App.js` (tree).
- Compose small components into bigger ones; use `{ }` to embed JS, `className` for CSS.

Now do `practice.md`.